Testing and Verification Guide: KubePilot

1. Test Suite Architecture

KubePilot incorporates a comprehensive automated test suite spanning unit tests, integration tests, API contract tests, and regression safety tests. The test suite is organized into 9 dedicated test modules containing **102 automated test cases**.

```
tests/
■■■ test_telemetry.py           # 12 tests: Prometheus ingestion, fallback, mocks
■■■ test_policy_engine.py       # 14 tests: YAML parsing, rule conditions, reset
■■■ test_decision_controller.py # 15 tests: PRAH arbitration, confidence scoring
■■■ test_guardrails.py          # 14 tests: Min/Max bounds, step limits, cooldowns
■■■ test_hysteresis.py          # 10 tests: Sliding window smoothing & flapping
■■■ test_kubernetes_executor.py # 11 tests: K8s client, scale patching, pod inspection
■■■ test_dashboard_api.py       # 13 tests: REST routes, health/readiness, status codes
■■■ test_experiments.py         #  6 tests: Workload generator & benchmark runner
■■■ test_fixes_regression.py    #  7 tests: Regression guards for resolved bugs
--------------------------------------------------------------------------------
Total: 102 Passing Automated Tests (100% Pass Rate)
```

2. Test Module Breakdown

2.1 `tests/test_telemetry.py`

Validates metric collection across live and mock operational states:

- Prometheus vector query string generation.
- Error handling during Prometheus timeout and unreachable network states.
- Fallback to direct Kubernetes Metrics API (**`metrics.k8s.io`**).
- Offline mock telemetry generation for deterministic CI testing.
- Schema conformance for metric snapshots (**`cpu`**, **`memory`**, **`requests_per_sec`**, **`latency_p95`**).

2.2 `tests/test_policy_engine.py`

Validates declarative policy evaluation and rule lifecycle:

- YAML syntax loading and schema validation (**`scaling-policies.yaml`**).
- Relational comparison operators (**`>`**, **`>=`**, **`<`**, **`<=`**).
- Target replica calculations (absolute, additive offsets, and proportional equations).
- Verification that **`reset()`** correctly clears rule registries without leaving stale state.
- Handling of missing or partial metric dictionaries.

2.3 `tests/test_decision_controller.py`

Tests the core **Policy-Driven Reactive-Arbitration Heuristic (PRAH)**:

- Single rule recommendation arbitration.
- Multi-rule conflict resolution (e.g., CPU requesting scale-up while Memory requests scale-down).
- Directional availability bias: guaranteeing scale-up priority over scale-down.
- Confidence score calculation across metric margin intervals.
- Zero-action NOOP handling when no policy conditions are breached.

2.4 `tests/test_guardrails.py`

Verifies hard safety limits and rate constraints:

- **Min Replica Floor:** Absolute enforcement of **`min_replicas = 2`** under all scale-down requests.
- **Max Replica Ceiling:** Strict capping at **`max_replicas = 10`** even under extreme simulated surges.
- **Step-Up Clamping:** Ensuring no single scaling action adds more than **`+3`** pods.
- **Step-Down Clamping:** Ensuring no single scaling action removes more than **`-1`** pod.
- **Cooldown Timers:** Verifying rejection of scale actions when cooldown windows are active (60s scale-up, 180s scale-down).

2.5 `tests/test_hysteresis.py`

Evaluates anti-oscillation and temporal smoothing logic:

- 5-sample sliding window metric evaluation.
- Rejection of premature scale-down actions caused by instantaneous single-tick metric dips.
- Detection and suppression of high-frequency directional reversals (flapping).

2.6 `tests/test_kubernetes_executor.py`

Tests Kubernetes cluster mutation and state inspection:

- Kubeconfig discovery and API client initialization.
- Auto-reconnection logic when cluster endpoint becomes temporarily unreachable.
- Successful JSON patch submission to deployment scale subresource.
- Extraction of pod status phases (**`Running`**, **`Pending`**, **`Failed`**) and readiness flags.

2.7 `tests/test_dashboard_api.py`

Validates Flask web application endpoints:

- Route availability and 200 OK responses: **`/`**, **`/workloads`**, **`/telemetry`**, **`/policies`**, **`/decisions`**, **`/history`**, **`/system-health`**.
- JSON REST API contracts: **`/api/status`**, **`/api/metrics`**, **`/api/decisions`**, **`/api/policies`**, **`/api/history`**.
- Kubernetes health probes: **`/healthz`** (200 OK) and **`/readyz`** (200 OK when policies loaded).
- Error handling and custom 404 template rendering.

2.8 `tests/test_experiments.py`

Tests the benchmarking framework:

- Parameter validation for **`stable_workload`** and **`traffic_spike`**.
- Execution loop across **`baseline`** HPA and **`proposed`** KubePilot modes.
- Metric time-series sampling and CSV/JSON output generation.

2.9 `tests/test_fixes_regression.py`

Guarantees against regression of critical project bug fixes:

- Regression test for maintaining the 2-pod baseline replica floor.
- Regression test verifying cooldown lockouts are logged with explicit rationale.
- Regression test confirming removal of deprecated experiment views from the overview page.

3. Running the Test Suite

3.1 Execute Full Test Suite

Run the entire test suite using Pytest:

```
python -m pytest tests/ -v
```

Expected output:

```
============================= test session starts =============================
platform win32 -- Python 3.13.2, pytest-9.1.1, pluggy-1.6.0
collected 102 items

tests/test_dashboard_api.py .............                                [ 12%]
tests/test_decision_controller.py ...............                        [ 27%]
tests/test_experiments.py ......                                        [ 33%]
tests/test_fixes_regression.py .......                                  [ 40%]
tests/test_guardrails.py ..............                                 [ 53%]
tests/test_hysteresis.py ..........                                     [ 63%]
tests/test_kubernetes_executor.py ...........                           [ 74%]
tests/test_policy_engine.py ..............                              [ 88%]
tests/test_telemetry.py ............                                    [100%]

============================= 102 passed in 1.48s =============================
```

3.2 Execute Individual Test Suites

```
# Run PRAH decision controller tests
python -m pytest tests/test_decision_controller.py -v

# Run guardrail and safety tests
python -m pytest tests/test_guardrails.py -v

# Run regression fix tests
python -m pytest tests/test_fixes_regression.py -v
```

3.3 Test Coverage Measurement

Measure statement coverage across the controller and dashboard codebases:

```
python -m pytest --cov=controller --cov=dashboard tests/
```

4. Live Cluster Verification Checklist

In addition to automated unit tests, perform this operational checklist against the running Minikube cluster:

Verification Item	Command / URL	Expected Result	Verified Status
Namespace Exists	**`kubectl get ns autoscaling-demo`**	Status **`Active`**	PASS
Sample App Running	**`kubectl -n autoscaling-demo get deployment sample-web-app`**	**`2/2`** Ready Replicas	PASS
Prometheus Active	**`curl -s http://localhost:9090/-/healthy`**	**`Prometheus Server is Healthy.`**	PASS
Dashboard UI	**`http://localhost:5055`**	200 OK, Cluster Online	PASS
Readiness Probe	**`curl -s http://localhost:5055/readyz`**	**`{"status": "ready", "policies_count": 6}`**	PASS
Decision Audit Log	**`Get-Content logs/decisions.jsonl`**	Structured JSONL records present	PASS
2-Pod Floor Protection	Trigger low-load scaling	Target clamped at *ge 2*	PASS
Step-Up Rate Limit	Trigger 100% CPU surge	Step capped at *+3* pods	PASS

API Reference: KubePilot REST APIs

1. Overview & Base URL

KubePilot exposes a comprehensive RESTful API for cluster state inspection, telemetry retrieval, policy management, scaling trigger execution, and experiment execution.

- **Base URL:** `http://localhost:5055`
- **Content-Type:** `application/json`
- **Authentication:** Unauthenticated (designed for internal cluster / operator access)

2. Cluster & Telemetry Endpoints

2.1 Get System Status

Returns the operational health of the control plane, Kubernetes connectivity, target deployment replica status, and Prometheus integration state.

- **Method:** `GET`
- **Path:** `/api/status`
- **Response `200 OK`:**

```
{
  "kubernetes": {
    "status": "connected",
    "context": "minikube",
    "namespace": "autoscaling-demo",
    "deployment": "sample-web-app",
    "desired_replicas": 2,
    "ready_replicas": 2,
    "available_replicas": 2,
    "updated_at": "2026-10-24T08:00:00Z"
  },
  "prometheus": {
    "status": "connected",
    "url": "http://localhost:9090",
    "reachable": true
  },
  "controller": {
    "status": "running",
    "mode": "embedded",
    "reconciliation_interval_seconds": 10,
    "active_policies_count": 6
  }
}
```

2.2 Get Current Telemetry Metrics

Returns the latest instantaneous metric snapshot aggregated from Prometheus and the cluster API.

- **Method:** `GET`
- **Path:** `/api/metrics`
- **Response `200 OK`:**

```
{
  "timestamp": "2026-10-24T08:00:10Z",
  "cpu_utilization_percent": 0.43,
  "memory_utilization_percent": 10.77,
  "requests_per_second": 0.62,
  "latency_p95_ms": 2020.25,
  "error_rate_5xx_percent": 0.0,
  "telemetry_source": "prometheus",
  "is_stale": false
}
```

3. Decision & Policy Endpoints

3.1 Get Latest Scaling Decisions

Returns the most recent scaling evaluation result produced by the PRAH engine, including triggered rules and guardrail validation status.

- **Method: `GET`**
- **Path: `/api/decisions`**
- **Response `200 OK`:**


```
{
  "decision_id": "dec-20261024-080010",
  "timestamp": "2026-10-24T08:00:10Z",
  "current_replicas": 2,
  "recommended_replicas": 2,
  "approved_replicas": 2,
  "action": "HOLD",
  "confidence_score": 1.0,
  "reason": "Current metrics within normal operating bounds",
  "triggered_rules": [],
  "guardrail_status": {
    "approved": true,
    "min_replicas_enforced": 2,
    "max_replicas_enforced": 10,
    "cooldown_active": false
  }
}
```


3.2 Get Active Scaling Policies

Returns the full set of declarative scaling policies loaded into memory from **`policies/scaling-policies.yaml`**.

- **Method: `GET`**
- **Path: `/api/policies`**
- **Response `200 OK`:**


```
{
  "policies": [
    {
      "name": "HighCPUUtilization",
      "metric": "cpu_utilization_percent",
      "operator": ">",
      "threshold": 80.0,
      "priority": 90,
      "target_replicas_calc": "proportional",
      "action": "scale_up"
    },
    {
      "name": "HighLatencySpike",
      "metric": "latency_p95_ms",
      "operator": ">",
      "threshold": 500.0,
      "priority": 85,
      "target_replicas_calc": "offset_+2",
      "action": "scale_up"
    },
    {
      "name": "LowResourceDrain",
      "metric": "cpu_utilization_percent",
      "operator": "<",
      "threshold": 20.0,
      "priority": 30,
      "target_replicas_calc": "offset_-1",
      "action": "scale_down"
    }
  ],
  "count": 3
}
```

3.3 Get Scaling Execution History

Returns historical scaling decisions loaded from the append-only **`logs/decisions.jsonl`** audit file.

- **Method: `GET`**
- **Path: `/api/history`**
- **Query Parameters:**
- **`limit`** (integer, optional): Maximum records to return (default: **`50`**).
- **Response `200 OK`:**

```
{
  "history": [
    {
      "timestamp": "2026-10-24T07:44:33Z",
      "from_replicas": 2,
      "to_replicas": 2,
      "action": "HOLD",
      "reason": "Scale down rejected: minimum floor 2 enforced",
      "applied": false
    }
  ],
  "total_records": 1
}
```

3.4 Force Single Decision Evaluation Tick

Triggers an immediate, synchronous evaluation tick across the telemetry, policy engine, and guardrails without waiting for the next periodic cycle.

- **Method: `POST`**
- **Path: `/api/evaluate`**
- **Response `200 OK`:**

```
{
  "status": "evaluated",
  "decision": {
    "target_replicas": 2,
    "action": "HOLD",
    "applied": false
  }
}
```

4. Workloads & Pod Inspection Endpoints

4.1 Get Workload Pod Information

Returns real-time container states, pod phase, restarts, and IP distribution for pods in the target deployment.

- **Method: `GET`**
- **Path: `/api/workloads`**
- **Response `200 OK`:**

```
{
  "deployment": "sample-web-app",
  "namespace": "autoscaling-demo",
  "pod_count": 2,
  "pods": [
    {
      "name": "sample-web-app-7984f8846c-k5zpw",
      "phase": "Running",
      "ready": true,
      "restart_count": 0,
      "pod_ip": "10.244.0.12",
      "node": "minikube"
    },
    {
      "name": "sample-web-app-7984f8846c-m9xqr",
      "phase": "Running",
      "ready": true,
      "restart_count": 0,
      "pod_ip": "10.244.0.13",
      "node": "minikube"
    }
  ]
}
```


5. Experiment Execution Endpoints

5.1 Run Benchmark Experiment

Launches an asynchronous benchmark run comparing Baseline HPA against KubePilot.

- **Method: `POST`**
- **Path: `/api/experiments/run`**
- **Request Body:**

```
{
  "experiment": "stable_workload",
  "mode": "both",
  "duration_seconds": 60
}
```

- **Response `202 Accepted`:**


```
{
  "status": "started",
  "run_id": "exp-stable_workload-20261024-080500",
  "message": "Benchmark experiment initiated"
}
```


5.2 Get Benchmark Experiment Results

Retrieves the completed benchmark comparison data and time-series metrics.

- **Method: `GET`**
- **Path: `/api/experiments/results`**
- **Query Parameters:**
- **`run_id`** (string, optional): Specific run ID. Defaults to latest completed run.
- **Response `200 OK`:**


```
{
  "run_id": "exp-stable_workload-20261024-074249",
  "status": "COMPLETED",
  "baseline_result": { "avg_cpu_percent": 0.38, "successful_requests": 10 },
  "proposed_result": { "avg_cpu_percent": 0.40, "successful_requests": 10 }
}
```

6. Kubernetes Health & Liveness Probes

6.1 Liveness Probe

- **Method: `GET`**
- **Path: `/healthz`**
- **Response `200 OK`:**

```
{
  "status": "healthy",
  "uptime_seconds": 3600
}
```

6.2 Readiness Probe

- **Method: `GET`**
- **Path: `/readyz`**
- **Response `200 OK`:**

```
{
  "status": "ready",
  "policies_count": 6
}
```

- **Response `503 Service Unavailable`:** Returned if policies have not been loaded or the cluster connection is uninitialized.